FOCAL SEMICON · POST-QUANTUM RETROFIT · EMBEDDED LINUX TRACK

Know your project, start to finish

Read top to bottom once. Each section ends with a line you can say out loud. Then use the practice questions to test yourself.

START HERE

The 60-second pitch

"Quantum computers will break the public-key cryptography in today's TLS: RSA, ECDH and ECDSA. I retrofitted a RISC-V board, the StarFive VisionFive 2, to use **hybrid post-quantum TLS 1.3**: ML-KEM-768 plus X25519 for the key exchange, and ML-DSA-44 plus ECDSA in place of RSA for server authentication."

"At the board's default settings, hybrid was about 30% slower than the legacy connection. I found three software-only fixes: raising the minimum CPU clock, compiling with the RISC-V bit-manipulation extensions, and checking the post-quantum signature on a second core. With all three, the full hybrid connection takes **14.5 ms vs 21.0 ms for the legacy baseline, 31% faster**. Against the hackathon server it's 7% faster. That's 1000 runs per mode, 0 failures, and only +1.3 °C at idle."

> IF YOU ONLY REMEMBER ONE THING
>
> Hybrid PQC is now faster than the old legacy baseline on this board, and I got there without new hardware.

FOUNDATIONS

Why post-quantum, and why now

Today's public-key cryptography rests on two hard maths problems: **factoring** large numbers (RSA) and the **discrete logarithm** on elliptic curves (ECDH, X25519, ECDSA). A large quantum

computer running **Shor's algorithm** solves both efficiently. That breaks every public-key algorithm in a normal TLS connection.

Symmetric cryptography (AES, SHA-2/3) is only weakened, by **Grover's algorithm**, which roughly halves the security level. AES-256 stays safe. That's why the retrofit only touches the public-key parts.

Harvest now, decrypt later

An attacker can record encrypted traffic today and decrypt it once a quantum computer exists. So the **key exchange** is urgent now: any data that must stay secret for years is already at risk. **Authentication** is less urgent: a forged signature only helps an attacker during a live connection, so it must be fixed before quantum computers arrive, but old recordings can't be attacked that way.

Why embedded devices matter

Industrial and IoT devices stay in the field for 10–20 years. A device shipped today will still be running when quantum attacks may be practical. Embedded CPUs are slow, so the worry is that PQC makes connections too slow. That's exactly what this project tests.

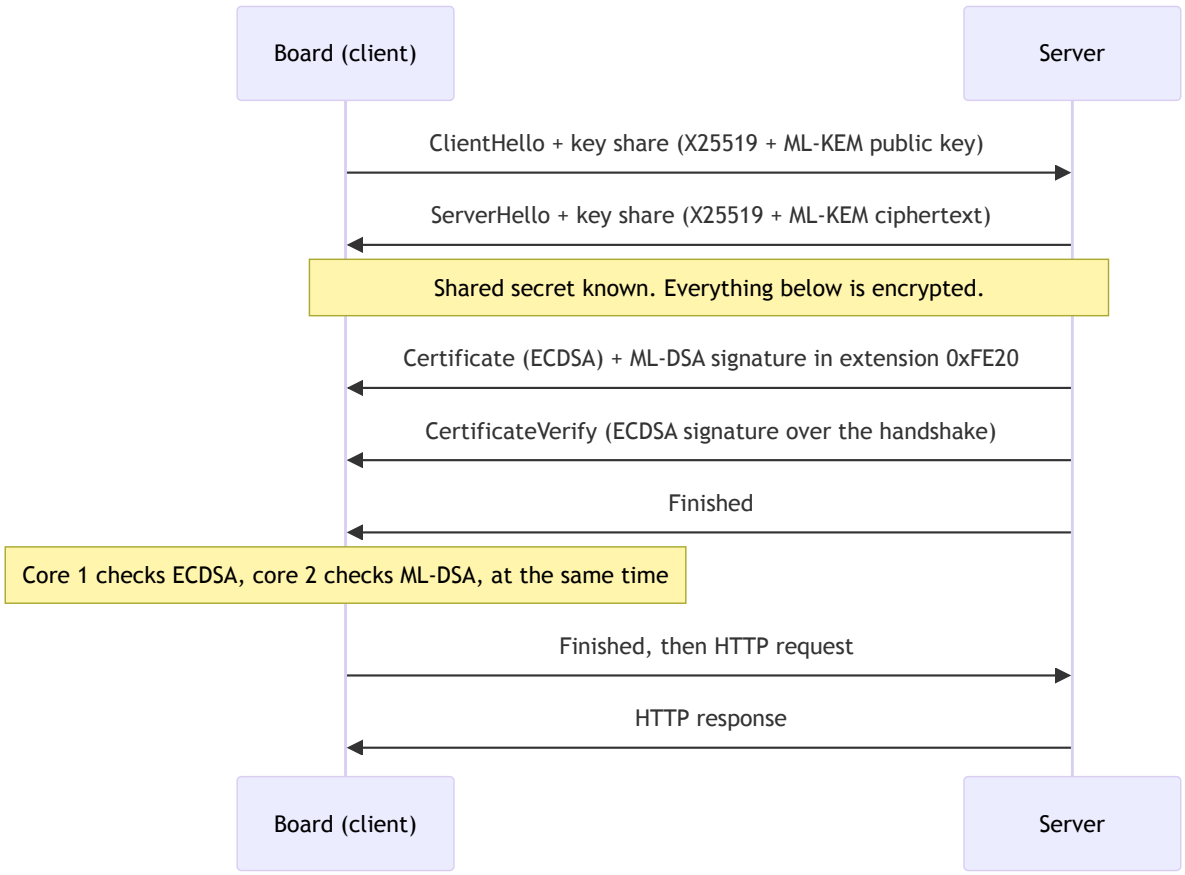
> SAY IT
>
> "Shor's algorithm breaks RSA and elliptic curves. Recorded traffic can be decrypted later, so the key exchange has to move to post-quantum now, and long-lived embedded devices have to move first."

FOUNDATIONS

How a TLS 1.3 connection works

A TLS handshake does two separate jobs. Keep them apart in your head, because the project upgrades each one differently.

1. **Key exchange:** both sides agree on a shared secret that nobody listening can work out. Legacy uses X25519 (elliptic-curve Diffie–Hellman).
2. **Authentication:** the server proves it is the real server, by signing the handshake with the private key that belongs to its certificate. The hackathon server uses RSA-3072.



TLS 1.3 needs **one round trip** before the client can send data. That's why the internet round-trip time (about 25–35 ms to the hackathon server) dominates there, while on the local network (about 1 ms) the CPU time of the crypto dominates. That second case is the hardest test for PQC.

> SAY IT
>
> "TLS does two jobs: key exchange and authentication. I made both hybrid: ML-KEM with X25519 for the key, and ML-DSA with ECDSA for the server's identity."

FOUNDATIONS

The algorithms, side by side

ML-KEM (FIPS 203, formerly CRYSTALS-Kyber) and **ML-DSA** (FIPS 204, formerly CRYSTALS-Dilithium) were standardised by NIST in August 2024. Both are **lattice-based**: their security rests on the Module Learning With Errors problem, solving noisy linear equations over polynomials, which no known quantum algorithm solves efficiently.

They work on polynomials with 256 coefficients, modulo a small prime: q = 3329 for ML-KEM and q = 8380417 for ML-DSA. The heavy work is polynomial multiplication via the **NTT** (a number-theoretic FFT) and generating matrices with **SHAKE / Keccak** hashing. That's why the SHA-3 speed-up from bit manipulation helped them so much.

Algorithm	Job	Quantum-safe?	Sizes	Cost on the bo
X25519	Key exchange	No	`32 B key`	very cheap
ML-KEM-768	Key exchange (KEM)	Yes	`1184 B pub key, 1088 B ciphertext`	`keygen+dec`
RSA-3072	Signature	No	`384 B sig`	`sign 34 ms`
ECDSA P-256	Signature	No	`≈71 B sig`	`verify 4.1`
ML-DSA-44	Signature	Yes	`1312 B pub key, 2420 B sig`	`sign 4.7 r`
ML-DSA-65	Signature	Yes	`3309 B sig`	`verify 1.6`

KEM vs Diffie–Hellman

A **KEM** (key encapsulation mechanism) works differently from Diffie–Hellman. The client generates a key pair and sends the public key. The server *encapsulates*: it creates a random secret and locks it with that public key, sending back the ciphertext. The client *decapsulates* with its private key. Both now hold the same 32-byte secret. It still fits in one round trip, inside the normal key-share fields.

Why ML-DSA-44 and not ML-DSA-65

ML-DSA-44 is NIST security category 2, a match for P-256's roughly 128-bit classical strength. The IETF composite-signature draft pairs it with P-256 for the same reason. It's also smaller (2420 vs 3309 B) and verifies faster (1.0 vs 1.6 ms). Switching saved 1.6 ms.

> SAY IT
> "ML-KEM and ML-DSA are the NIST lattice standards from 2024. ML-KEM-768 is category 3, ML-DSA-44 is category 2, matched to P-256."

FOUNDATIONS

Why hybrid instead of pure PQC

The new algorithms are young. If someone finds a classical attack on ML-KEM next year, a pure-PQC system would be broken today. Hybrid combines a classical and a post-quantum algorithm so that **an attacker has to break both**:

- **Hybrid key exchange (X25519MLKEM768):** the two shared secrets are concatenated and fed into TLS's key schedule. The session key stays safe as long as *either* half holds. This is the same group Chrome, Firefox and Cloudflare already use by default.
- **Hybrid authentication (dual signature):** the client accepts the server only if *both* the ECDSA and the ML-DSA signatures verify. A forger must break both.

Hybrid is also the transition path regulators recommend (for example Germany's BSI and France's ANSSI), and it keeps the classical algorithms that certifications still require.

> SAY IT
> "Hybrid means an attacker must break both the classical and the post-quantum part. If ML-KEM turns out weak, X25519 still protects us, and the reverse."

YOUR HARDWARE

The StarFive VisionFive 2

- **SoC:** StarFive JH7110, 4 × SiFive U74 cores, **RV64GC**, up to 1.5 GHz, Debian Linux.
- **No vector unit** (no RVV), so no wide SIMD for the NTT. It does support the **Zba and Zbb** bit-manipulation extensions, which the default compiler settings don't use.
- **All 4 cores share one clock** (one frequency policy). You can't run one core fast and the others slow.

- **Default clock policy:** the `ondemand` governor, lowest step 312.5 MHz. It raises the clock only after it notices load.
- **Hailo-8L:** an int8 neural-network accelerator over PCIe. It wasn't detected on the board, and the analysis says it wouldn't help anyway (see the questions).
- Connected to the Mac by USB serial console and Ethernet. The board reaches the internet through the Mac.

> SAY IT
>
> "Four in-order RISC-V cores, no vector unit, one shared clock. So the tools I had were the clock policy, the compiler and the spare cores."

YOUR WORK

What you built

1. A modern TLS library on the board

Debian ships OpenSSL 3.0.9, which has no ML-KEM or ML-DSA. You built **OpenSSL 3.5.9** (the long-term-support release with both algorithms built in) from source on the board, in a separate folder so the system library stays untouched. Three builds:

Build	Change	Effect
std	plain OpenSSL 3.5.9	ECDSA verify slow (4.1 ms): generic C code on RISC-V
opt	`enable-ec_nistp_64_gcc_128`	Fast, constant-time P-256 code: ECDSA verify 4.1 → 1.4 ms
bitmanip	opt + `-march=rv64gc_zba_zbb`	SHA-3 +50%, ML-KEM −13 to −19%, ML-DSA verify −15%

2. Hybrid key exchange

The client asks for the TLS group `X25519MLKEM768`. Against the hackathon server, its reply `"handshake_method":"hybrid-pqc"` confirmed it.

3. Hybrid authentication, two versions

The hackathon server only signs with RSA, so you wrote your own test server (on the Mac). It has an RSA certificate for legacy clients and an ECDSA certificate plus ML-DSA keys for hybrid clients. Both modes were measured against this same server, so the comparison is fair.

- **v1:** after the handshake, the server signs a value unique to this connection (the TLS exporter) with ML-DSA and sends it with the first HTTP response. It works and needs no extra round trip, but the check can only start after the handshake, so it adds time.
- **v2 (final):** the ML-DSA signature travels *inside the Certificate message* as a custom TLS extension (`0xFE20`). It signs a hash of the ClientHello and ServerHello, which contain both sides' random values and key shares, so it can't be replayed in another connection. Because it arrives together with the ECDSA signature, the board checks **both at the same time on two cores**.

4. Trust model

The client **pins** the server's certificate and ML-DSA public key, instead of trusting a public certificate authority. That's fine for a closed device fleet. For the open internet you'd use a PKI with post-quantum certificates (see Limits).

Security checks you ran

A wrong ML-DSA key is rejected. A missing ML-DSA signature is rejected. A wrong server certificate is rejected. The client never sends application data until both signatures have passed.

> SAY IT
>
> "I built OpenSSL 3.5 on the board. The key exchange is the standard X25519MLKEM768 group. For authentication I added an ML-DSA-44 signature next to ECDSA, inside the TLS handshake, bound to that exact connection, and both must pass."

YOUR EVIDENCE

How you measured

- **Baseline** = the original legacy connection, X25519 + RSA-3072, at the board's default CPU setting.
- **1000 connections per mode.** Each comparison ran in 20 blocks of 50, alternating which mode goes first. Network changes or chip temperature then affect both modes equally.

- **Median** as the headline number, because internet tails are huge (worst cases up to 524 ms from one network hiccup). The mean would be dragged around by a few outliers. Best, worst, p95 and mean are all in the reports too.
- **Paired blocks:** in each block the two modes are compared directly. "Faster in 20/20 blocks" is very strong evidence. If there were no real difference, the chance of that is about 1 in a million. 17/20 corresponds to $p \approx 0.003$.
- **Two scenarios:** your own server on the local network (crypto dominates, the hard case) and the hackathon server over the internet (the network dominates, the realistic case).
- **Micro-benchmarks at a fixed 1.5 GHz.** Under `ondemand` the clock changed between runs and timings varied 2×. That noisy file is kept and labelled unreliable.
- **Every run checked:** the hybrid group was actually negotiated, and ML-DSA actually verified. 0 failures.

> WATCH OUT
>
> The very first numbers (165 ms legacy vs 64–79 ms hybrid) were wrong: a local relay program added its own connection time to the legacy runs. If asked, say you caught it, fixed it, and that's why you switched to large, alternating, paired runs.

> SAY IT
>
> "1000 runs per mode, alternated in blocks so both modes saw the same conditions. I report medians and the number of blocks where one mode won."

YOUR SOLUTION

The three speed-ups, and why each works

At the default setting, hybrid was **27.95 ms vs 21.61 ms legacy** (+29%) on your server, and +13% on the hackathon server. Each fix attacks a different cause.

Approach 1: Raise the minimum CPU clock (biggest win)

Problem: `ondemand` lets the CPU idle at 312.5 MHz and only speeds up after it notices load. A handshake is a 20 ms burst, so it's mostly over before the clock ramps up. The crypto runs at a fifth of full speed.

Fix: keep `ondemand` (still saves power) but raise `scaling_min_freq` to **750 MHz**. One settings change, no code.

Why 750 MHz: you tested 750 and 1250 MHz minimums, the `schedutil` governor, and full speed. 750 MHz was as fast as 1250 MHz but cooler (+0.5 °C idle vs +1.9 °C). Fixed 1.5 GHz is fastest but runs about +6 °C hotter, which you saw heat the chip. `schedutil` at the default minimum was *worse* than the default.

Result alone: hybrid 27.84 → 16.19 ms (own server).

Approach 2: RISC-V bit-manipulation extensions

Problem: the U74 supports Zba and Zbb, but standard builds target plain RV64GC. Without them a 64-bit rotate takes three instructions (shift left, shift right, OR).

Fix: recompile OpenSSL with `-march=rv64gc_zba_zbb`. No code changes. The compiler uses single-instruction rotates (`rol`/`ror`), `andn`, and shifted adds (`sh3add`).

Why it helps PQC: Keccak (SHA-3/SHAKE) is mostly 64-bit rotations and AND-NOT. ML-KEM and ML-DSA spend much of their time in SHAKE. So SHA-3 got 50% faster and ML-KEM 13–19% faster. RSA, ECDSA and X25519 didn't change.

Result alone: hybrid 28.15 → 26.57 ms (−5.6%, 20/20 blocks). Small, but free.

Catch: the binary only runs on CPUs with Zba/Zbb.

Approach 3: Use a second core

Problem: hybrid authentication does two independent checks (ECDSA and ML-DSA), but normally one after another on one core while three cores sit idle.

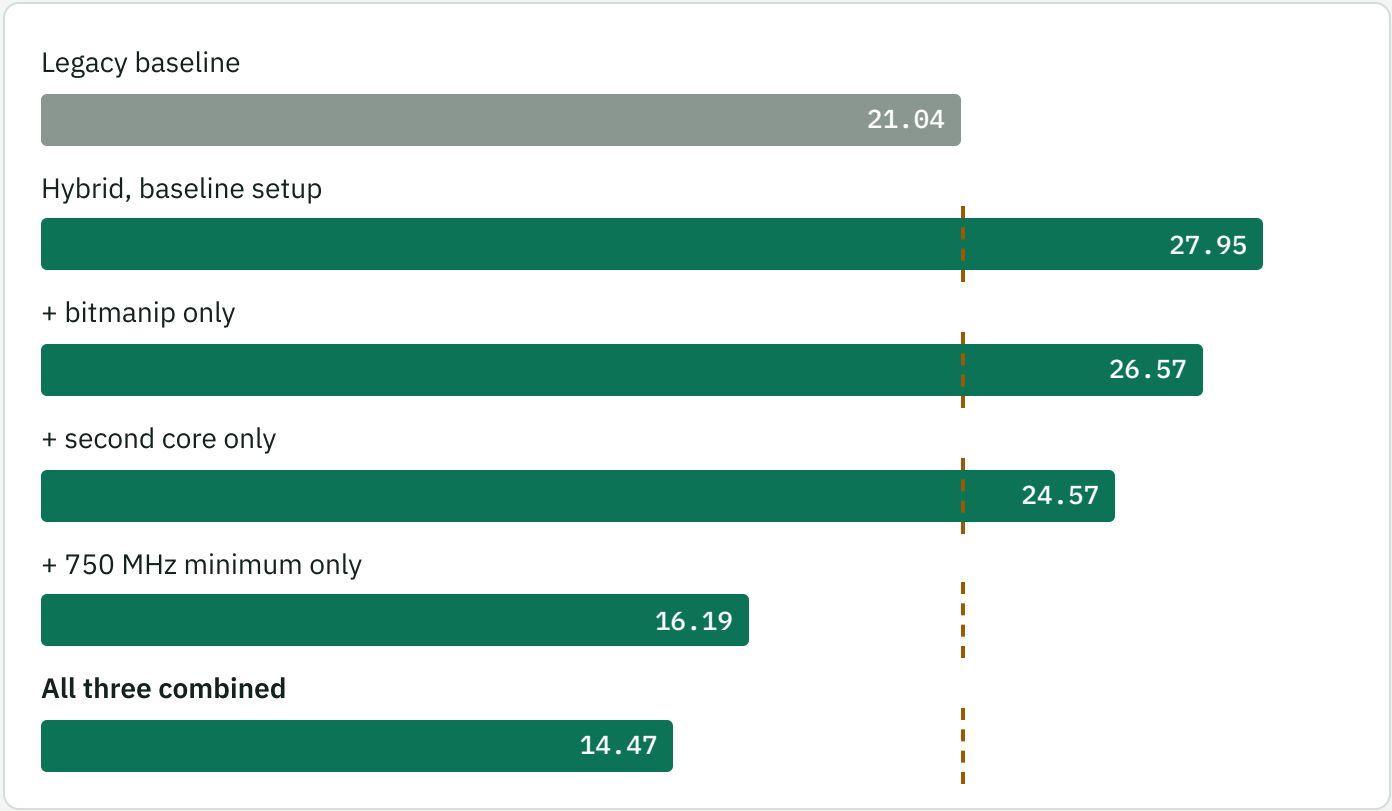
Feasibility test first: a separate program timed each pair of classical and PQ operations, on one core vs two. The result was 1.5–1.65× faster, and client crypto went from 16.0 to 10.1 ms at the default clock. It uses a **persistent worker thread**, because creating a new thread per operation costs too much.

Into the real handshake: that's why v2 moved the ML-DSA signature into the Certificate message. When it arrives, a worker thread on core 2 verifies ML-DSA while OpenSSL verifies ECDSA on core 1.

Result alone: hybrid 28.19 → 24.57 ms (−12.8%, 20/20 blocks), no extra heat.

Why not 4 cores: there are only two independent checks. The rest of the handshake is a strict sequence (Amdahl's law).

How they add up (own server, hybrid median)



Milliseconds, scale 0–30 ms. The dashed line is the legacy baseline. Single-approach rows come from separate runs, so treat them as indicative.

> SAY IT
>
> "The default clock was the biggest problem, so a 750 MHz floor gave the biggest win. The compiler flag speeds up SHA-3 inside the PQ algorithms. The second core runs the two signature checks in parallel. Together they make hybrid 48% faster."

YOUR RESULT

Final results (all approaches combined)

BASE (default setup) and COMBINED (all three fixes) were alternated block by block in one session: 8000 connections, 0 failures.

Scenario	Legacy baseline	Hybrid, BASE	Hybrid, COMBINED	vs legacy baseline
Own server, full hybrid (end-to-end)	**21.04 ms**	**28.06 ms**	**14.47 ms**	**−6.57 ms (−31%), 20/20**
Hackathon server, hybrid key exchange (handshake)	**38.36 ms**	**43.76 ms**	**35.62 ms**	**−2.73 ms (−7%), 17/20**
Idle temperature	45.9 °C	45.9 °C	47.2 °C	+1.3 °C

Three facts judges may push on

- **Why hybrid gains more than legacy:** the same fixes take hybrid down 48% but legacy only 27%. The fixes speed up client-side crypto, and hybrid does more of it.
- **Like for like, hybrid is even 1 ms faster than legacy on your server** (both at COMBINED, 20/20 blocks). That's because the server's RSA-3072 signing (several ms) is slower than ECDSA + ML-DSA-44 signing.
- **On the hackathon server, like for like, hybrid is still +2.8 ms slower.** The bigger ML-KEM messages cost a little on the internet, and hybrid authentication can't be used there. Your claim is about beating the *baseline*, which is what the task asks for.

> SAY IT
>
> "Task: no increase over the legacy baseline. Result: hybrid is 31% faster with full hybrid TLS on my server, and 7% faster against the hackathon server, for about one degree more at idle."

TASK-SHEET METRICS

Memory, communication, energy, acceleration

Metric	Legacy	Hybrid	What causes it
Peak memory (client process)	`6.3–6.6 MB`	`+0.1–0.2 MB`	Bigger keys and buffers; negligible
Bytes received	`2,520 B`	`5,105 B`	ML-DSA-44 signature 2,420 B + ML-KEM ciphertext 1,088 B
Bytes sent	`330 B`	`1,552 B`	ML-KEM-768 public key 1,184 B in the ClientHello
Resumed connection	`4.60 ms`	`5.98 ms`	No signatures; fresh hybrid key exchange (full speed, day 1)

- **Communication:** about +3.8 KB per handshake. That's small for Ethernet. It matters for very constrained links (LoRa, NB-IoT), where the 1.2 KB ClientHello may need several packets.
- **Sending signatures as raw bytes** instead of base64 saved about 800 B.
- **Energy:** not measured, because you had no USB-C power meter. Temperature is the proxy: +1.3 °C idle with COMBINED, the same during the runs, vs about +5–6 °C at full speed.
- **Acceleration (Hailo-8L):** not used and not detected on PCIe. The reasoning is in the practice questions.

BE UPFRONT

Limits (say these before they ask)

- **Hybrid authentication only against your own server.** The hackathon server only signs with RSA-3072, so against it only the key exchange is hybrid.
- **The ML-DSA signature uses a custom TLS extension,** not a standard. In production you'd use standard-track options: composite ML-DSA certificates (IETF LAMPS draft), pure ML-DSA certificates, or "Catalyst" certificates with an alternative key.
- **Pinned keys, not a PKI.** That's fine for a closed device fleet, but a real rollout needs CA-issued post-quantum certificates.
- **The bitmanip build only runs on CPUs with Zba/Zbb.**
- **The key exchange halves still run one after another** inside OpenSSL. Parallelising them would need an OpenSSL patch (another 0.5–1 ms).

- **Energy wasn't measured.**
- **The 750 MHz minimum isn't permanent yet.** It needs a boot service, like the one used for full speed earlier.
- **Server is a Mac,** not a second embedded device.

> SAY IT
> "The key exchange is fully standard. My authentication method is a working prototype; for production, I'd move it to composite ML-DSA certificates once the standard is final."

SELF-TEST

Practice questions

Answer out loud first, then open the card. Tick the ones you know; your ticks are saved in this browser.

BASICS

– **Why do we need post-quantum cryptography?**

Shor's algorithm on a large quantum computer breaks RSA and elliptic-curve crypto, which is every public-key algorithm in today's TLS. Recorded traffic can be decrypted later (harvest now, decrypt later), so key exchange must move now.

– **Is AES broken by quantum computers?**

No. Grover's algorithm only roughly halves the security level. AES-256 stays safe. Only the public-key parts need replacing.

– **What's the difference between ML-KEM and ML-DSA?**

ML-KEM (FIPS 203) agrees on a shared secret: it replaces the ECDH key exchange. ML-DSA (FIPS 204) is a signature: it replaces RSA/ECDSA for proving the server's identity. Both are lattice-based NIST standards from August 2024.

- **Why hybrid and not pure post-quantum?**

  The PQ algorithms are new. If a classical attack is found, pure PQC is broken. Hybrid needs both parts broken: the session key is safe if either X25519 or ML-KEM holds, and the server can only be impersonated if both ECDSA and ML-DSA are forged. Regulators recommend hybrid for the transition.

- **Which is more urgent, key exchange or authentication?**

  Key exchange. Recorded traffic can be decrypted later. A signature can only be forged during a live connection, so authentication must be fixed before quantum computers exist, but old recordings aren't at risk through it.

- **How is a KEM different from Diffie–Hellman?**

  Client sends a public key; the server encapsulates a random secret with it and returns the ciphertext; the client decapsulates. Both end up with the same secret. It still fits in TLS 1.3's single round trip.

DESIGN

- **Why ML-DSA-44 and not ML-DSA-65 or 87?**

  ML-DSA-44 is NIST category 2, matching P-256's ~128-bit classical strength. The IETF composite draft pairs them the same way. It's smaller (2420 vs 3309 B) and verifies faster (1.0 vs 1.6 ms). The key exchange uses ML-KEM-768 (category 3) because that's the standard TLS hybrid group.

- **Why ECDSA instead of keeping RSA alongside ML-DSA?**

  The task asked to replace RSA. ECDSA is the faster, smaller classical partner: signing 0.64 ms vs 34 ms for RSA-3072, signature 71 vs 384 B. Pairing it with ML-DSA keeps the total server signing cost below RSA alone.

- **How do you stop someone replaying a stolen ML-DSA signature?**

  It signs a hash of this connection's ClientHello and ServerHello. Those contain fresh random values and the key shares from both sides, so the signature only matches this one handshake. (v1 signed the TLS exporter, which is also unique per connection.)

- **What if an attacker strips the ML-DSA extension?**

  The client requires it. A missing signature is rejected (tested), and the client sends no application data until both signatures pass.

- **Why not liboqs / oqs-provider or a different TLS library?**

  OpenSSL 3.5 is a long-term-support release with ML-KEM and ML-DSA built in, using the final FIPS versions. That's fewer moving parts and closer to what a real product would ship. oqs-provider is mainly for research; it adds a plugin and more algorithms we didn't need.

- **Why use a custom extension instead of a standard method?**

  No standard composite ML-DSA certificate format was final, and OpenSSL 3.5 can't yet do dual signatures in one handshake. The custom extension let me prove the performance and the parallel check today. The production path is composite ML-DSA certificates (IETF LAMPS).

- **Why pin keys instead of using certificate authorities?**

  For a closed device fleet, pinning is simple and strong: the device trusts only its own server. A public rollout would need post-quantum certificates from a CA. That's a deployment step, not a performance one.

PERFORMANCE

- **Why was hybrid slower at the default setting?**

  The CPU idles at 312.5 MHz and the ondemand policy only speeds up after it notices load; a 20 ms handshake is mostly over before that. Hybrid has more client-side crypto (ML-KEM, ECDSA + ML-DSA verify), so the slow clock hurt it more: +6.4 ms on the LAN.

- **Why is 750 MHz the best minimum clock?**

  Of the tested settings, it was as fast as 1250 MHz and schedutil at 750, but the coolest of them (+0.5 °C idle). Full 1.5 GHz is fastest but about +6 °C hotter. 500 and 625 MHz weren't tested; that's a possible follow-up.

- **Why not simply run the CPU at full speed?**

  I did on day 1: it gives 10.3 ms vs 10.7 ms legacy. But the chip ran hot (about 50–52 °C idle), which is bad for a device in an enclosure, and it wastes power when idle. The 750 MHz floor gets most of the benefit for about a fifth of the heat.

- **What exactly do Zba and Zbb do, and why do they help PQC?**

  Zbb adds rotate (rol/ror), AND-NOT and bit-count instructions; Zba adds shifted adds for address arithmetic. Keccak/SHAKE, used heavily inside ML-KEM and ML-DSA, is mostly 64-bit rotates and AND-NOT, which took 3 instructions each before. So SHA-3 got 50% faster and ML-KEM 13–19% faster.

- **Why does bitmanip barely change the hackathon-server number?**

  There, the ~30 ms network round trip dominates and the crypto is a small slice, so a 1–2 ms gain disappears in network noise. On the LAN it's clearly measurable: −1.6 ms, 20/20 blocks.

- **How does the multicore approach work, and is it safe?**

  A persistent worker thread on a second core verifies ML-DSA while OpenSSL verifies ECDSA on the main core. Before any application data is sent, the main thread waits for the worker; if either check fails, the connection is dropped. Verification uses only public data, so running it on another thread leaks nothing.

- **Why only two cores and not all four?**

  There are only two independent checks (classical and PQ). The rest of the handshake is a strict sequence of messages, so extra cores have nothing to do (Amdahl's law). The key-exchange halves could also run in parallel, but that needs an OpenSSL patch.

- **Why is a persistent thread better than starting a thread per handshake?**

  Creating a thread costs time on this CPU. The feasibility test compared both, and the persistent worker was the best option, so the handshake code uses it.

- **Why does hybrid beat legacy like-for-like on your server?**

  The server signs: RSA-3072 signing takes several milliseconds, while ECDSA + ML-DSA-44 signing is cheaper. Once client-side crypto is fast, that server-side difference wins: hybrid is 1 ms faster, 20/20 blocks.

- **Would a device that signs (mutual TLS) do better or worse with PQC?**

  Better. On the board, ECDSA + ML-DSA-44 signing takes 0.64 + 4.69 ≈ 5.3 ms, vs 34 ms for RSA-3072.

METHOD

- **Why median and not average?**

  Internet runs have huge outliers (up to 524 ms from one hiccup). The mean jumps around with a few of them; the median shows the typical connection. All stats (best, p95, mean, worst) are in the reports.

- **How do you know the improvement isn't luck or network noise?**

  1000 runs per mode, alternated in 20 blocks so both saw the same conditions, and compared per block. Hybrid won 20 of 20 blocks on the LAN: by chance that's about 1 in a million. 17/20 on the internet is $p \approx 0.003$.

- **Why does your final 14.5 ms differ from the 10.3 ms on day 1?**

  Day 1 ran at a fixed 1.5 GHz. The final result uses the 750 MHz floor with ondemand, which is cooler. Each comparison was measured against a legacy baseline under the matching conditions.

- **What happened with the first 165 ms legacy measurement?**

  It was a measurement error: a local relay added its own connection time to the legacy runs. I found it, removed it, and that's why I moved to large alternating and paired runs.

TASK SHEET

- **What's the communication overhead?**

  About +3.8 KB per handshake: received 5,105 vs 2,520 B (ML-DSA signature + ML-KEM ciphertext), sent 1,552 vs 330 B (ML-KEM public key). Raw signatures instead of base64 saved about 800 B.

- **What's the memory overhead?**

  Peak process memory is 6.3–6.6 MB in all modes; hybrid adds only about 0.1–0.2 MB.

- **Did you measure energy?**

  No: I had no USB-C power meter. Temperature is my proxy: +1.3 °C idle with the combined setup, no difference while running, vs about +5–6 °C at full speed. Measuring energy per handshake is the first follow-up.

- **Why didn't you use the Hailo-8L accelerator?**

  It wasn't detected on PCIe. Even if it were, it's an 8-bit neural-network chip: ML-KEM works mod 3329 (12-bit) and ML-DSA mod 8380417 (23-bit), so the maths would need splitting and extra steps. All the PQ crypto is only a few ms per handshake, and the PCIe round trip would eat a large part of any gain. The CPU-side fixes delivered more.

- **What would you put in a migration guide?**

  1) Upgrade to OpenSSL 3.5 in a separate prefix. 2) Check the server supports hybrid groups and PQ signatures. 3) Set a 750 MHz CPU floor. 4) Build with enable-ec_nistp_64_gcc_128 and -march=rv64gc_zba_zbb. 5) Use ML-DSA-44 with P-256. 6) Verify the two signatures in parallel. 7) Use session resumption for frequent reconnects.

LIMITS

- **What would you do with more time?**

  Parallelise the key-exchange halves inside OpenSSL; test 500/625 MHz floors; measure energy with a power meter; move authentication to standard composite certificates; make the 750 MHz floor a boot service; test on a real embedded server and constrained networks.

- **Is your solution portable to other RISC-V boards?**

  The clock floor and the second core work on any multicore Linux board. The bitmanip build needs Zba/Zbb; other chips just use the plain build and lose about 1.6 ms.

REFERENCE

Glossary

PQC

Post-quantum cryptography: algorithms believed safe against quantum computers.

ML-KEM

Module-Lattice Key Encapsulation Mechanism, FIPS 203 (was Kyber). Agrees on a shared secret.

ML-DSA

Module-Lattice Digital Signature Algorithm, FIPS 204 (was Dilithium). Proves identity.

SLH-DSA

FIPS 205, hash-based signatures (was SPHINCS+). Very large signatures; not used here.

X25519

Elliptic-curve Diffie–Hellman on Curve25519. Classical key exchange.

X25519MLKEM768

The hybrid TLS group: both key shares are sent, and both secrets are combined.

ECDSA P-256

Elliptic-curve signatures on the NIST P-256 curve. Classical.

RSA-3072

Classical signatures; slow to sign, fast to verify.

Shor / Grover

Quantum algorithms: Shor breaks RSA and elliptic curves; Grover only weakens symmetric keys.

NTT

Number-theoretic transform, a fast way to multiply polynomials; the core of ML-KEM and ML-DSA.

Keccak / SHAKE

The SHA-3 family; ML-KEM and ML-DSA use it heavily.

Zba / Zbb

RISC-V bit-manipulation extensions: address arithmetic, and rotates / AND-NOT / bit counting.

`ondemand`

Linux CPU clock policy that raises the clock after it sees load.

`scaling_min_freq`

The lowest clock the policy is allowed to use.

`Exporter`

A TLS function that gives a secret value unique to one connection (used by hybrid auth v1).

`Pinning`

The client trusts one specific certificate or key instead of a certificate authority.

`Median / p95`

The middle value / the value 95% of runs are below.

`RTT`

Round-trip time on the network.